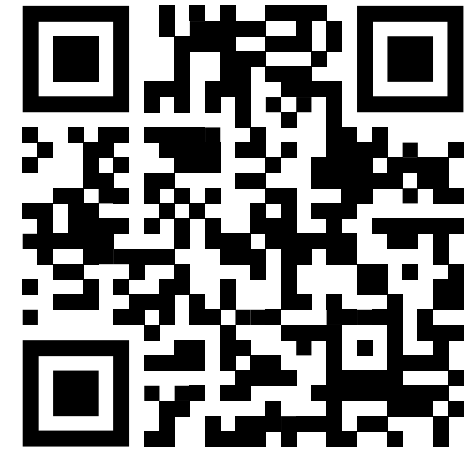


Programmierung in C++

Input & Output

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>

Input & Output

Überblick



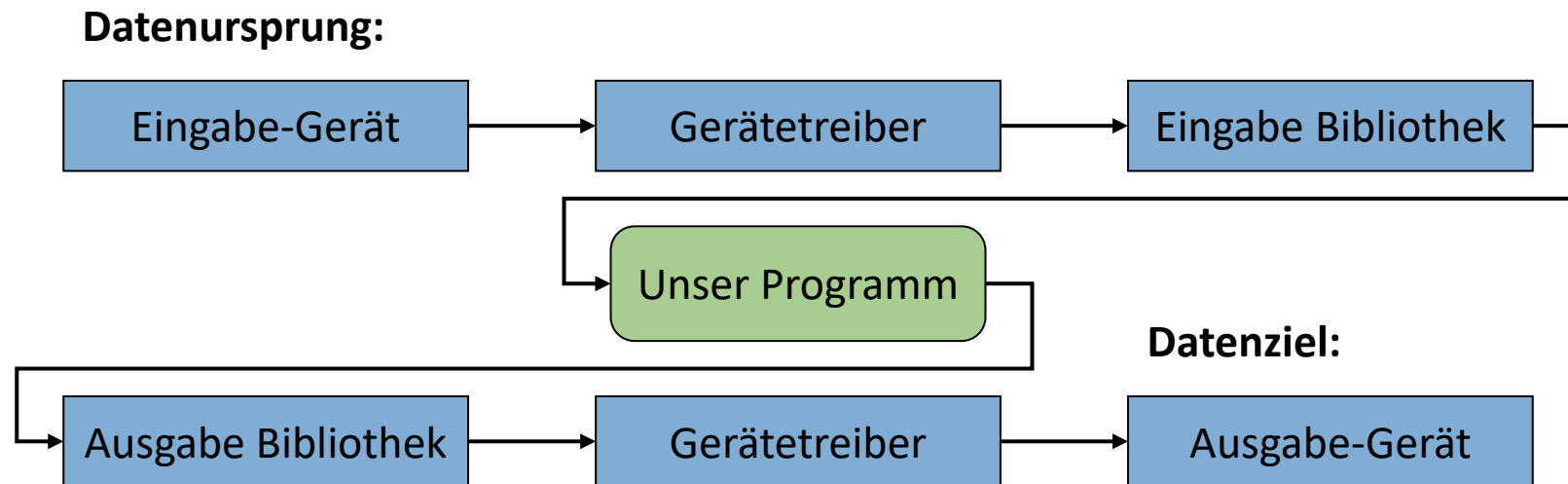
- Die I/O Stream Bibliothek der STL stellt formatierte und unformatierte Puffer für die Ein- und Ausgabe von Text und numerischen Werten zur Verfügung
 - Alle Klassen besitzen Destruktoren zur Freigabe von allokiertem Speicher (RAII)
 - Streams können nicht kopiert werden, sondern müssen als Referenz übergeben werden
 - Die STL bietet keine Funktionalitäten für grafische Ausgaben
- Die File-System Bibliothek verbessert die Arbeit mit Dateien und Ordnerstrukturen.
 - Bietet eine einheitliche Schnittstelle für unterschiedliche Ordnerstrukturen
 - Ermöglicht die Navigation durch eine Ordnerstruktur via C++ Code
- Die C++ Standard Bibliothek unterstützt auch die C-Style I/O-Operationen (*printf*, *scanf*).
 - Diese sind jedoch typunsicher und sicherheitskritisch
 - Die Kompatibilität mit den C-Style I/O-Operationen kann mit dem Aufruf `std::ios_base::sync_with_stdio(false);` ausgeschaltet werden, was zu bedeutenden Performanz-Verbesserungen führt.

Input & Output

Überblick



- Eine I/O Bibliothek stellt Schnittstellen zur Verfügung, mit denen sich alle Geräte (Bildschirme, Tastatur, etc.) möglichst gleich ansteuern lassen
 - Sonst müsste man für jedes Gerät (auch jeden unterschiedlichen Bildschirm) ein eigenes Programm schreiben
 - Die meisten modernen Betriebssysteme spalten das detaillierte Verhalten von Ein- und Ausgabegeräten in Gerätetreiber, auf die mithilfe der I/O Bibliothek zugegriffen wird
 - Mehr dazu in der Vorlesungsveranstaltung Betriebssysteme





Streams

Lernziele

- Die Funktionsweise bereits verwendeter Stream wurde verstanden.
- Mithilfe des Status von Streams können Überprüfungen getätigt und zuverlässigerer Code geschrieben werden.
- Weitere Arten von Streams sowie ihr Nutzen sind bekannt.

Streams

Überblick



- Standard Streams sind mit den Standard-I/O-Streams des Betriebssystems verknüpft
 - ***std::cout*** für gewöhnliche Konsolenausgaben
 - ***std::cerr*** für ungepufferte Fehlerausgaben
 - ***std::clog*** für gepufferte Log-Meldungen
 - ***std::cin*** für gewöhnliche Konsoleneingaben
- Die I/O-Stream Bibliothek definiert eine Ein- und Ausgabe für alle elementaren Datentypen

```
int main() {  
    std::clog << "Program started.\n";           // Output Log Stream  
    std::cout << "Wie heißt du?\n";              // Standard Output Stream  
    std::string name;  
    std::cin >> name;                             // Standard Input Stream  
    std::cout << "Hallo " << name << ".\n";      // Standard Output Stream  
    std::clog << "Program ended.\n";             // Output Log Stream  
}
```

Streams

Ausgabe-Stream



```
#include <ostream>
```

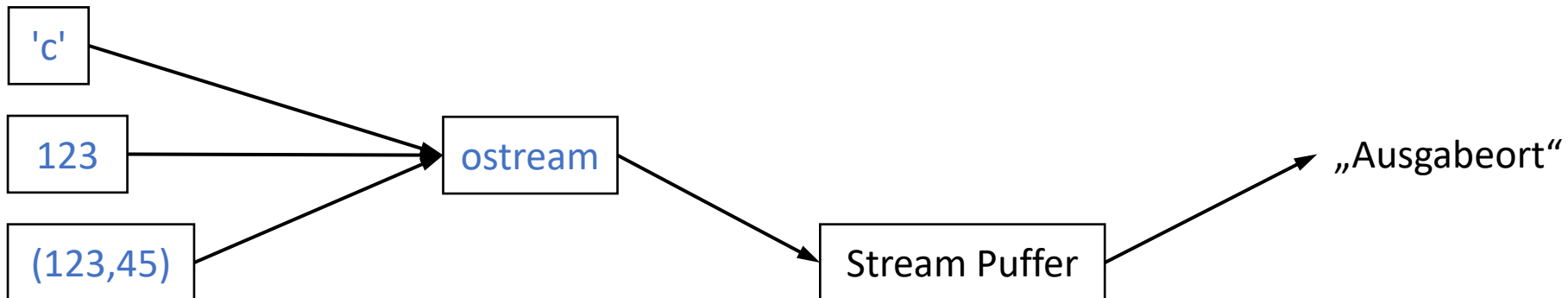
- Ein Ausgabe-Stream (**ostream**) konvertiert typisierte Objekte in einen Stream von Zeichen.
 - Über den Stream Puffer können weitere Zeichen erfasst werden, während der Ausgabe-Stream mit dem Betriebssystem kommuniziert
 - `std::ostream` ist ein alias für die Realisierung der Klasse `basic_ostream` mit `chars`

```
namespace std {  
    using ostream = basic_ostream<char>;  
}
```

Typisierte Objekte:



Byte Sequenzen:



Streams

Eingabe-Stream



```
#include <istream>
```

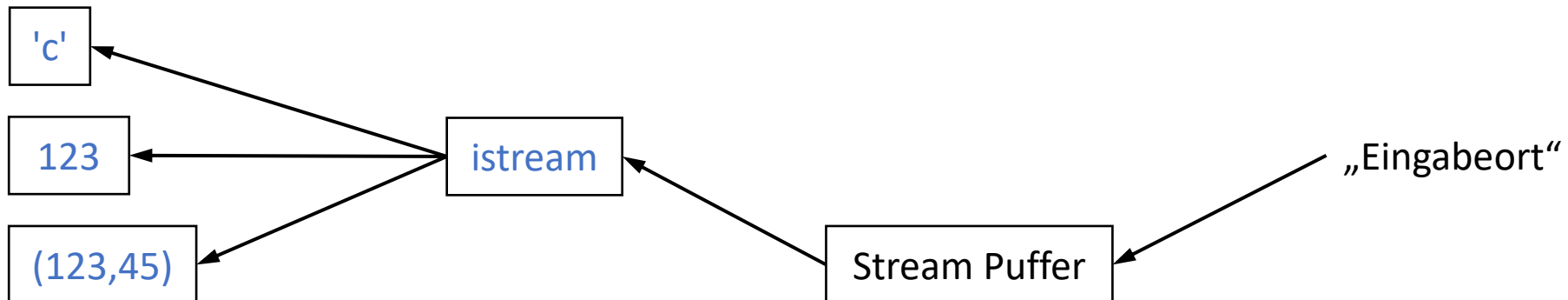
- Ein Eingabe-Stream (**istream**) konvertiert ein Stream von Zeichen in typisierte Objekte

- Verwendet den >> Operator für Eingaben und die Zuweisung zu Variablen
- Eine Eingabe ist standardmäßig durch ein Leerzeichen, einen Zeilenumbruch oder durch ein Zeichen, welches nicht dem Variablentyp entspricht, abgeschlossen.
- Eingaben können verkettet werden
- Ein Eingabe-Stream kann z.B. an *getline()* übergeben werden, um eine ganze Zeile zu lesen

```
std::string zeile;  
getline(std::cin, zeile);
```

Typisierte Objekte:

Byte Sequenzen:



Streams

I/O Status



- Ein- und Ausgabe-Streams besitzen einen Status, der angibt, ob eine Operation erfolgreich war
 - Findet häufig beim Einlesen von mehreren Werten Anwendung
 - Die Überprüfung auf einen I/O-Stream gibt True zurück, wenn der Stream für eine weitere Operation bereit ist.
 - Der Status enthält alle zum Lesen/Schreiben notwendigen Informationen (u.a. zu Formatierung, Fehler, Puffertyp)
 - Der Programmierer kann den Status ändern, um zum Beispiel Fehler mitzuteilen oder zu entfernen

```
std::vector<int> read_ints(std::istream& is)
{
    std::vector<int> res;
    for (int i; is >> i;)
        res.push_back(i);
    return res;
}
```

is >> gibt eine Referenz auf den Eingabe-Stream zurück, der so lange er Integer-Werte liest, für weitere Werte bereit ist.

Streams

I/O Status



- Ein- und Ausgabe-Streams besitzen einen Status, der angibt, ob eine Operation erfolgreich war
 - Findet häufig beim Einlesen von mehreren Werten Anwendung
 - Die Überprüfung auf einen I/O-Stream gibt True zurück, wenn der Stream für eine weitere Operation bereit ist.
 - Der Status enthält alle zum Lesen/Schreiben notwendigen Informationen (u.a. zu Formatierung, Fehler, Puffertyp)
 - Der Programmierer kann den Status ändern, um zum Beispiel Fehler mitzuteilen oder zu entfernen

```
std::vector<int> read_ints
(std::istream& is, const std::string& terminator)
{
    std::vector<int> res;
    for (int i; is >> i;)
        res.push_back(i);

    if (is.eof()) // Ende der Eingabe
        return res;
    if (is.fail()) { // Kein Integer eingeben
        is.clear(); // Status zu good() zurücksetzen
        std::string s;
        if (is>>s && s==terminator) // Zeichenkette war das Trennwort
            return res;
        is.setstate(std::ios_base::failbit); // Fehler setzen
    }
    return res;
}
```

eof ist das „end of file“-Flag und ist gesetzt, wenn keine weiteren Zeichen vorhanden sind.

Liest die Zeichenkette, die kein Integer war, nun in einen String.

Streams

String Streams



```
#include <sstream>
```

- Die Standard Bibliothek definiert Streams zu und von Strings
 - ***std::istream*** zum Lesen von einem *std::string*
 - ***std::ostream*** zum Schreiben zu einem *std::string*
 - ***std::stringstream*** zum Lesen und Schreiben
- Gängige Verwendung, wenn Zeichenketten formatiert werden sollen, bevor sie weitergegeben werden
 - Zum Beispiel, um Zahlen in einer anderen Schreibweise auszugeben

```
std::ostream oss;
```

```
oss << "{temperature," << std::scientific << 123.4567890 << "};"  
std::cout << oss.str() << '\n';
```

Gibt nachfolgende Zahlen in wissenschaftlicher Schreibweise aus.

```
{temperature,1.234568e+02}
```

Streams

String Streams



```
#include <sstream>
```

- String Streams können verwendet werden, um beliebige Datentypen, die sich als String darstellen lassen, ineinander umzuwandeln

```
template<typename Target = string, typename Source = string>
Target to(Source arg) { // wandelt Source in Target um
    std::stringstream buf;
    Target result;
    if (!(buf << arg) // Schreibe arg in den Stringstream
        || !(buf >> result) // Lies result aus dem Stringstream
        || !(buf >> std::ws).eof()) // Überprüfe, ob nicht alles umgewandelt werden konnte
        throw std::runtime_error{"to<>() failed!"};
    return result;
}
```

```
auto x1 = to<std::string,double>(1.2); // explizite Typangabe
auto x2 = to<std::string>(1.2); // Source wird zu double deduziert
auto x3 = to<>(1.2); // Target wird zu std::string defaulted; Source wird zu double deduziert
auto x4 = to(1.2); // Das <> ist optional, wenn kein Argument angegeben wird
```

Streams

C++23 Memory Streams

- Seit den Urzeiten von C++ gibt es Streams, die an nutzerdefinierten Speicherbereichen gebunden sind
 - Die ältesten dieser Streams (*std::stringstream*) sind schon lange überholt
 - C++23 führt den Ersatz für die veralteten Memory Streams ein:
std::spanstream, ***std::ispanstream*** und ***std::ospanstream***
 - Memory Streams und String Streams verhalten sich sehr ähnlich, nur dass Memory Streams einen Speicherbereich statt einem String als Argument erhalten
 - Wenn ein Pufferüberlauf auf dem Speicherbereich verursacht werden würde, wird stattdessen der Status des Streams auf Fehler gesetzt



```
#include <spanstream>
```

```
template<typename... param>
void user(param... args) {
    std::array<char,128> buf;
    std::ospanstream ss(buf);
    ss << "write ";
    (ss << ... << args) << " to memory\n";
    for (auto ch : buf)
        std::cout << ch;
}

int main() {
    user(356, ' ', 3.1415, ' ', "Hello");
}
```

Output:

```
write 356 3.1415 Hello to memory
0Thr0 0S00 0hr0 hQhr0 80S00 0Fr0
```

Streams



C++20 Synchronisierte Streams

```
#include <syncstream>
```

- In einem multi-threaded System können Ein- & Ausgaben ein wildes Chaos erzeugen, es sei denn
 - nur ein Thread kann auf den Stream zugreifen oder
 - Zugriff auf den Stream ist synchronisiert, sodass zu jeder Zeit nur ein Thread Zugriff besitzt
- ***std::osyncstream*** gewährleistet, dass die Ausgabeoperationen auch in einem multi-threaded System wie erwartet funktionieren
 - Dazu wird dem synchronisierten Stream ein anderer Stream übergeben, auf den thread-sicher zugegriffen werden soll
 - Alle Threads, die ausschließlich synchronisierte Streams verwenden, können sich nicht gegenseitig stören. Wenn ein Thread einen nicht synchronisierten Stream verwendet, kann dieser jedoch die synchronisierten Streams stören.

```
void saferOutput(int x, std::string& s) {  
    std::osyncstream oss(std::cout);  
    oss << x;  
    oss << s;  
}
```

Streams



Zusammenfassung

- Streams werden für die Ein- und Ausgabe verwendet
 - Es existieren Streams, die nur für die Eingabe oder nur für die Ausgabe implementiert sind.
 - Mithilfe von Stream Puffer werden Zeichen auch dann noch berücksichtigt, wenn der Stream mit dem Betriebssystem kommuniziert.
 - Streams sind für alle elementaren Datentypen überladen.
- Über den I/O Status lässt sich bestimmen, ob eine Ein- oder Ausgabe erfolgreich war.
 - Bei einem fehlerhaften Status, muss der Status eigenständig zurückgesetzt werden, bevor der Stream weiterverwendet werden kann.
- Für unterschiedliche Anwendungsfälle existieren unterschiedliche Streams
- Die Konsole ist eine geteilte Ressource. `std::cout` und `std::cin` sind daher nicht thread-sicher.
 - Für die Verwendung der Konsole in multi-threaded Systemen existieren synchronisierte Streams.

Streams

Verständnisfragen

PIN: 8008135

<https://poll.hs-kempten.de/poll/>



- **Welche dieser Standard-Streams sind gepuffert?**
 - `std::cout`
 - `std::clog`
 - `std::cerr`
 - `std::cin`
- **Was muss getan werden, um einen Eingabestream nach dem Lesen eines ungültigen Zeichens wiederzuverwenden?**
 - Fehlerstatus zurücksetzen mit `clear()`
 - Formatflags zurücksetzen mit `unsetf()`
 - Stream schließen und neu öffnen
 - Überreste aus dem Stream entfernen
- **WAHR oder FALSCH?**
 - Leerzeichen beenden das Einlesen mit `std::cin` in eine Variable.
 - `std::cin` ist mit dem Typ `std::istream` assoziiert.
 - Synchronisierte Streams gewährleisten die Thread-Sicherheit von Ausgaben.



Dateien & Verzeichnisse

Lernziele

- Dateien können gelesen, erstellt und beschrieben werden.
 - Dazu können unterschiedliche Modi verwendet werden.
- Ein sicherer Umgang mit Datei- und Verzeichnis- Pfaden ist möglich.
 - Verzeichnisse können erstellt und traversiert werden.
 - Auf Dateiendungen kann zugegriffen werden.
 - Dateien können kopiert und verschoben werden.

File Streams



Dateien

- Informationen können in Dateien auf Festplatten und Co. gespeichert werden
 - Dadurch kann deutlich mehr gespeichert werden als im Arbeitsspeicher möglich wäre
 - Die Informationen sind persistent, auch wenn der Strom ausgeschalten wird
 - Grundslegend besteht eine Datei aus einer Sequenz von Bytes, die von 0 aufsteigend nummeriert sind:
0: 1: 2:

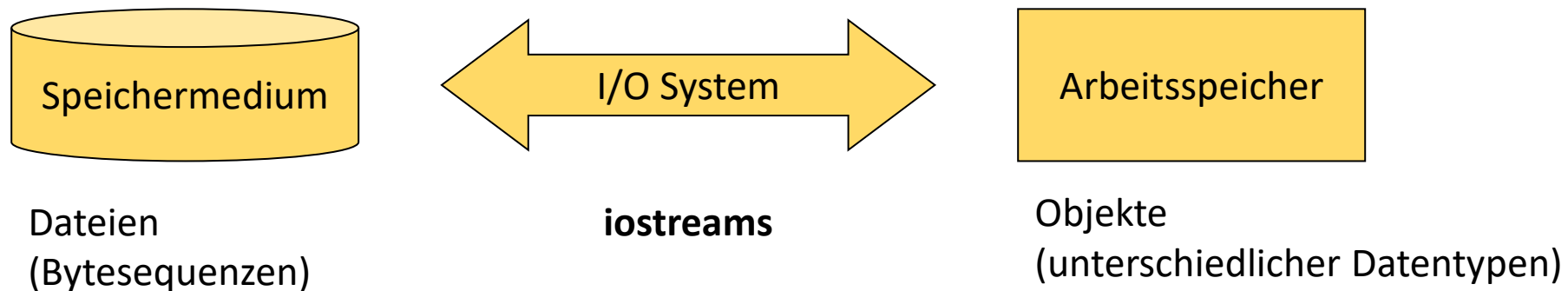
- Eine Datei besitzt ein bestimmtes Dateiformat, welches bestimmt, wie die Bytes interpretiert werden müssen
 - Das Dateiformat für Dateien auf Festplatten spielt eine ähnliche Rolle wie Datentypen für den Arbeitsspeicher
 - Für Textdateien könnte die Interpretation sein, dass die ersten vier Byte auch die ersten vier Zeichen sind.
 - In einem Dateiformat, welches Ganzzahlwerte binär codiert, könnten die ersten vier Byte die erste Ganzzahl sein

File Streams



Dateien

- Auch für Dateien existieren Streams mit denen Daten aus Dateien gelesen und in Dateien geschrieben werden können.
 - Der **Ausgabe-Stream** konvertiert Objekte vom Arbeitsspeicher in einen Stream von Bytes und schreibt sie auf das Speichermedium.
 - Der **Eingabe-Stream** liest einen Stream von Bytes aus dem Speichermedium und erzeugt ein Objekt im Arbeitsspeicher.
 - Dabei ist es egal, ob das Speichermedium eine Festplatte, ein USB-Stick, eine SD-Karte oder ein anderes Medium ist.



File Streams

Dateien



```
#include <fstream>
```

- Das **Lesen** aus einer Datei verhält sich ähnlich wie *std::cin*.
 - ***std::ifstream*** ist ein Stream zum Lesen von Daten aus einer Datei.
 - Um eine Datei zu lesen, müssen wir
 1. Den Namen der Datei kennen
 2. Die Datei (zum Lesen) öffnen
 3. Die Zeichen einlesen
 4. Die Datei schließen (geschieht typischerweise implizit)
- Das **Schreiben** in eine Datei verhält sich ähnlich wie *std::cout*.
 - ***std::ofstream*** ist ein Stream zum Schreiben von Daten in eine Datei.
 - Um in eine Datei zu schreiben, müssen wir
 1. Die Datei benennen
 2. Die Datei (zum Schreiben) öffnen oder eine Datei mit dem Namen erstellen
 3. Die Objekte in die Datei schreiben
 4. Die Datei schließen (geschieht typischerweise implizit)

File Streams

Dateien öffnen



```
#include <fstream>
```

- File Streams müssen zur Verwendung mit Dateien verknüpft sein
 - Idealerweise wird ein File Stream direkt mit dem Dateipfad **initialisiert**
 - Öffnet die Datei, die mit dem String des Pfades übereinstimmt
 - Schließt die Datei automatisch wieder, wenn die Lebenszeit des Streams beendet ist

Nach dem Verknüpfen der Datei, muss überprüft werden, ob diese korrekt geöffnet wurde.

Beim Verlassen des Umgebungsbereichs wird die Datei automatisch geschlossen.

```
{
    std::cout << "Gib den Namen der Datei ein: ";
    std::string fileName;
    std::cin >> fileName;

    std::ifstream fileStream {fileName};
    if (!fileStream)
        throw std::runtime_error("can't open input file " + fileName);

    vector<int> numbers;
    for (int i; fileStream >> i; )
        numbers.push_back(i);
}
```

fileStream ist ein Eingabe-Stream für die Datei mit dem Namen fileName

Aus der Datei können Zahlen gelesen werden wie mit std::cin aus der Nutzereingabe.

File Streams

Dateien öffnen



```
#include <fstream>
```

- File Streams müssen zur Verwendung mit Dateien verknüpft sein
 - Alternativ kann dem Stream auch nach der Initialisierung eine Datei mit ***open()*** verknüpft werden.
 - Der Stream kann vor der Verknüpfung **nicht** verwendet werden.
 - Die Datei **muss** manuell mit der ***close()*** Operation wieder geschlossen werden.
 - Bietet sich nur dann an, wenn die Lebenszeit der Verknüpfung nicht geeignet durch den Umgebungsbereich limitiert wird.

```
std::ifstream fileStream;  
fileStream >> number;                                // Fehler: Für fileStream ist keine Datei geöffnet  
  
fileStream.open(fileName, std::ios_base::in); // Öffnet die Datei namens fileName zum Lesen  
if (!fileStream)  
    throw std::runtime_error("can't open input file " + fileName);  
  
fileStream.close();                                // Schließt die Datei  
fileStream >> number;                                // Fehler: Die Datei für fileStream wurde geschlossen
```

File Streams

Dateien öffnen



```
#include <fstream>
```

- File Streams müssen zur Verwendung mit Dateien verknüpft sein
 - Idealerweise wird ein File Stream direkt mit einem String **initialisiert**
 - Öffnet die Datei, die mit dem String übereinstimmt
 - Schließt die Datei automatisch wieder, wenn die Lebenszeit des Streams beendet ist
 - Alternativ kann dem Stream auch nach der Initialisierung eine Datei mit ***open()*** verknüpft werden.
 - Der Stream kann vor der Verknüpfung nicht verwendet werden.
 - Die Datei muss manuell mit der ***close()*** Operation wieder geschlossen werden.
 - Bietet sich nur dann an, wenn die Lebenszeit der Verknüpfung nicht geeignet durch den Umgebungsbereich limitiert wird.
- Wird eine Datei geschlossen, so wird der verknüpfte Ausgabe-Puffer „**geflushed**“: Das heißt, die noch zu schreibenden Zeichen im Puffer werden vor dem Schließen noch in die Datei geschrieben. (Sie werden nicht verworfen!)

File Streams

Dateien öffnen



```
#include <fstream>
```

- Der **Modus**, mit dem eine Datei geöffnet wird, kann weiter spezifiziert werden.
 - Wird optional als weiterer Parameter nach dem Dateinamen angegeben
 - Mit einem bitweisen Oder können mehrere Modi angegeben werden

File Stream Öffnungs-Modi	
<code>ios_base::app</code>	„append“: Fügt Daten nur am Ende der Datei hinzu
<code>ios_base::ate</code>	„at the end“: Öffne und springe zum Ende
<code>ios_base::binary</code>	Öffne als Binärdatei (besitzt Betriebssystem-spezifisches Verhalten)
<code>ios_base::in</code>	Öffne zum Lesen
<code>ios_base::out</code>	Öffne zum Schreiben
<code>ios_base::trunc</code>	„truncate“: entferne den bisherigen Inhalt

```
// Default ist ios_base::out ->
std::ofstream of1 {name1};
// Default ist ios_base::in ->
std::ifstream if1 {name2};
// ofstream besitzt automatisch ios_base::out ->
std::ofstream ofs {name, std::ios_base::app};
// ios_base::in und ios_base::out ->
std::fstream fs {"myfile",
                 std::ios_base::in | std::ios_base::out};
```

File Streams

Lesen & Schreiben



```
#include <fstream>
```

■ Telefonbucheinträge in eine Datei **schreiben**

```
struct Entry {  
    std::string name;  
    uint32_t number = 0;  
    bool operator<(const Entry& other) const {  
        return name == other.name  
            ? number < other.number  
            : name < other.name;  
    }  
};  
  
class Phonebook {  
public:  
    // Erstellt ein Telefonbuch aus einer Liste an Einträgen  
    Phonebook(std::initializer_list<Entry> entries);  
    // Speichert den Inhalt des Telefonbuchs in einer Datei  
    bool save(const std::string& fileName);  
private:  
    std::set<Entry> m_entries;  
};
```

Phonebook.h

```
#include "Phonebook.h"  
Phonebook::Phonebook(std::initializer_list<Entry> entries)  
    : m_entries{ entries } {}  
bool Phonebook::save(const std::string& fileName) {  
    std::ofstream file{ fileName, std::ios_base::app };  
    if (!file) return false;  
  
    for (const auto& [name, number] : m_entries)  
        file << name << " " << number << "\n";  
    return true;  
}
```

Phonebook.cpp

```
Phonebook book{  
    {"David Hume", 123456},  
    {"Karl Popper", 234567},  
    {"Bertrand Russell", 345678}  
};  
book.save("phonebook.txt");
```

int main()

phonebook.txt

```
Bertrand Russell 345678  
David Hume 123456  
Karl Popper 234567
```


File Streams

Lesen & Schreiben



```
#include <fstream>
```

■ Telefonbucheinträge aus der Datei **auslesen**

```
struct Entry {  
    std::string name;  
    uint32_t number = 0;  
    bool operator<(const Entry& other) const { /* ... */ }  
};  
  
class Phonebook {  
public:  
    // Erstellt Telefonbuch anhand Einträge einer Datei  
    explicit Phonebook(const std::string& fileName);  
    // Liest Telefonbucheinträge aus einer Datei  
    bool load(const std::string& fileName);  
  
private:  
    std::set<Entry> m_entries;  
};
```

```
// Lies Telefonbuch aus Datei  
int main()  
{  
    Phonebook book{ "phonebook.txt" };  
}
```

Phonebook.h

```
#include "Phonebook.h"  
Phonebook::Phonebook(const std::string& fileName) {  
    if (!load(fileName))  
        std::cerr << "File could not be read!\n";  
}  
  
bool Phonebook::load(const std::string& fileName) {  
    std::ifstream file{ fileName };  
    if (!file) return false;  
    Entry readEntry;  
    // Telefonbucheinträge immer mit Vor- und Nachnamen  
    std::string firstName, lastName;  
    while (file >> firstName >> lastName >> readEntry.number)  
    {  
        readEntry.name = firstName + " " + lastName;  
        m_entries.emplace(readEntry);  
    }  
    return true;  
}
```

Phonebook.cpp

File Streams

Lesen & Schreiben

- Telefonbucheinträge aus der Datei auslesen und **ausgeben**



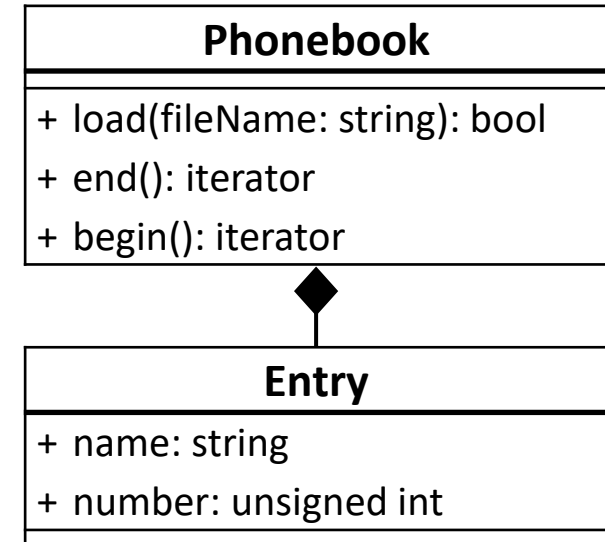
```
#include <fstream>
```

```
// ...
std::set<Entry>::const_iterator Phonebook::begin() const { return m_entries.begin(); }
std::set<Entry>::const_iterator Phonebook::end() const { return m_entries.end(); }

std::ostream& operator<<(std::ostream& os, const Entry& e) {
    return os << e.name << ": " << e.number;
}

std::ostream& operator<<(std::ostream& os, const Phonebook& pb) {
    for (const auto& entry : pb)
        os << entry << '\n';
    return os;
}
```

Phonebook.cpp



phonebook.txt

```
Bertrand Russell 345678
David Hume 123456
Karl Popper 234567
```

int main()

```
Phonebook book{ "phonebook.txt" };
std::cout << book;
```

```
Bertrand Russell: 345678
David Hume: 123456
Karl Popper: 234567
```

File Streams

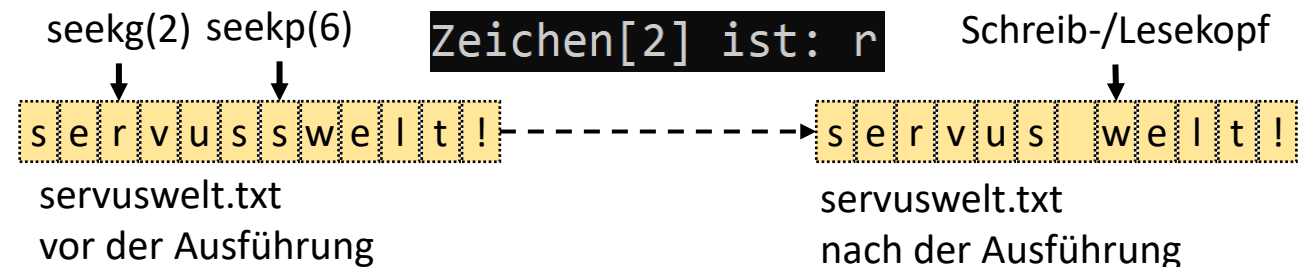
Lese- & Schreib- Positionen

- Wann immer möglich sollten Dateien von Anfang bis Ende gelesen oder beschrieben werden.
 - Das ist der einfachste und am wenigsten fehleranfällige Weg.
 - Selbst, wenn Änderungen an einer Datei getätigt werden müssen, ist es meistens sinnvoller eine neue Datei mit den Änderungen zu erstellen.
 - Wenn unbedingt notwendig, kann die Position des Schreib-/Lesekopfes auf ein bestimmtes Zeichen gesetzt werden.



```
#include <fstream>
```

```
std::fstream fs{ "servuswelt.txt" };  
fs >> std::noskipws; // Lies Leerzeichen mit  
if (fs) {  
    char ch;  
    // S/L-Kopf zu Position 2 und "get" das Zeichen  
    fs.seekg(2) >> ch;  
    std::cout << "Zeichen[2] ist: " << ch << '\n';  
  
    // S/L-Kopf zu Position 6 und "put" das Zeichen  
    fs.seekp(6) << ' ';  
}
```



File Streams

Binärdateien

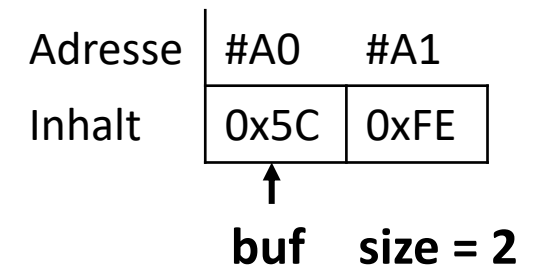


```
#include <fstream>
```

- Binärdateien enthalten die Roh-Daten von Objekten so wie sie Arbeitsspeicher stehen würden.
 - Zahlen werden nicht in ihre entsprechenden Symbole umgewandelt.
 - Daten im Binär-Format sind nicht menschenlesbar.
 - Für elementare Datentypen wird immer ein fester Platzbedarf benötigt.
- Da keine Typ-Umwandlung oder -Formatierung notwendig ist, werden die Text-orientierten Operatoren << und >> bei Binärdateien für gewöhnlich **nicht** verwendet.
 - Stattdessen werden die Funktionen *read()* und *write()* verwendet, die File-Streams von I/O-Streams erben.

```
istream& istream::read(char* buf, int size);  
ostream& ostream::write(const char* buf, int size);
```

```
short num = -420; :
```



File Streams

Binärdateien



```
#include <fstream>
```

- Binärdateien enthalten die Roh-Daten von Objekten so wie sie Arbeitsspeicher stehen würden.
 - Verwendung der Funktionen *read()* und *write()*, die File-Streams von I/O-Streams erben.

```
unsigned int i = 123456;  
{  
    std::ofstream file{ "data.bin", std::ios::binary };  
    file.write(reinterpret_cast<const char*>(&i), sizeof(i));  
}
```

Angabe, dass es sich um eine Binärdatei handelt.

Um die Roh-Daten des Integer zu verwenden, müssen wir den Zeiger auf den Int als Zeiger auf ein Zeichen interpretieren.

Mithilfe von `sizeof()` erhalten wir die Anzahl Bytes des Objekts.

data.bin (Inhalt als hexadezimale Bytes angezeigt)

Offset	+0	+1	+2	+3	+4	+5	+6	+7	+8	ASCII Zeichen
0000	40	e2	01	00						@

```
std::ifstream file{ "data.bin", std::ios::binary };  
file.read(reinterpret_cast<char*>(&i), sizeof(i));  
  
std::cout << i;
```

123456

File Streams

Binärdateien



```
#include <fstream>
```

- Bei komplexeren Datentypen ist es wichtig, die Daten binär so abzuspeichern, dass sie auch wieder eindeutig eingelesen werden können.

```
struct Values {  
    int i;  
    double d;  
};
```

Values val{42, 3.14};

```
std::ofstream file{ "data.bin", std::ios::binary };  
file.write(reinterpret_cast<const char*>(&val), sizeof(Values));
```

Schreiben

```
std::ifstream file{ "data.bin", std::ios::binary };  
file.read(reinterpret_cast<char*>(&val), sizeof(Values));
```

Lesen

- Bei Objekten mit beliebiger Größe muss die Größe gespeichert werden.

```
std::string str{ "Hello World" };
```

```
std::ofstream file{ "data.bin", std::ios::binary };  
int size = str.size();  
file.write(reinterpret_cast<const char*>(&size), sizeof(size));  
file.write(str.c_str(), size);
```

Schreiben

```
std::ifstream file{ "data.bin", std::ios::binary };  
int size;  
file.read(reinterpret_cast<char*>(&size), sizeof(size));  
file.read(str.data(), size);
```

Lesen

Der in `std::string::data()` vorhandene Speicherplatz muss groß genug sein, um alle Zeichen aufnehmen zu können.

C++17 File System



```
#include <filesystem>
```

- Die File System Bibliothek der STL ist eine Schnittstelle unabhängig vom Betriebssystem, um
 - Pfade auszudrücken und durch die Verzeichnisse zu navigieren
 - Dateiformate und Zugriffsrechte zu betrachten
- Die Klasse ***std::filesystem::path*** kann mit den unterschiedlichen Zeichenmengen und Konventionen vieler Betriebssysteme umgehen
 - Akzeptiert auch Befehlszeilenargumente als Eingabe
 - Validität wird erst bei Verwendung überprüft

Die Befehlszeilenargumente können u.a. durch das Starten des Programms im Konsolenfenster gesetzt werden.

Anzahl
Befehlszeilenargumente

Array der
Befehlszeilenargumente

```
F:\projects\P2Sandbox\x64\Debug>start P2Sandbox.exe file.txt
```

argv[0]

argv[1]

```
int main(int argc, char* argv[]) {  
    if (argc < 2) { std::cerr << "Arguments expected\n"; return 1; }  
    std::filesystem::path p{ argv[1] }; // Erzeugt einen Pfad aus dem Befehlszeilenargument  
    std::cout << p << " " << std::boolalpha << std::filesystem::exists(p) << '\n';  
    std::getchar();  
}
```

```
F:\projects\P2Sandbox\x64\Debug\P2Sandbox.exe  
"file.txt" true
```

C++17 File System

Pfade



```
#include <filesystem>
```

File System Typen (unvollständig)

path	Ein Ordnerpfad
filesystem_error	Eine File System Exception
directory_entry	Ein Objekt der Ordnerstruktur
directory_iterator	Zum Iterieren über ein Verzeichnis
recursive_directory_iterator	Zum Iterieren über ein Verzeichnis und seine Unterverzeichnisse

```
using namespace std::filesystem;
void print_directory(path p)
try {
    if (is_directory(p)) {
        std::cout << p << "\n"; // Gib den Pfad aus
        // Gib die Elemente im Verzeichnis aus
        for (const directory_entry& x : directory_iterator{ p })
            std::cout << "\t" << x.path() << '\n';
    }
} catch (const filesystem_error& ex) {
    std::cerr << ex.what() << '\n';
}
```

```
int main() {
    // aktuelles Verzeichnis
    print_directory(".");
    // Elternverzeichnis
    print_directory("..");
    // UNIX Root-Verzeichnis
    print_directory("/");
}
```

```
".":
    ".\\P2Sandbox.cpp"
    ".\\P2Sandbox.vcxproj"
    ".\\P2Sandbox.vcxproj.filters"
    ".\\P2Sandbox.vcxproj.user"
    ".\\pch.cpp"
    ".\\pch.h"
    ".\\x64"

"..":
    "..\\.vs"
    "..\\P2Sandbox"
    "..\\P2Sandbox.sln"
    "..\\x64"

"/":
    "$RECYCLE.BIN"
    "Config.Msi"
    "DeliveryOptimization"
    "desktop.ini"
    "Program Files"
    "Programme"
    "projects"
    "System Volume Information"
    "Windows10Upgrade"
    "WindowsApps"
    "WpSystem"
```


C++17 File System

Pfade

- Die Klasse ***path*** bietet viele nützliche Funktionalitäten
 - ***p/=p2*** konkateniert den Pfad p2 mit dem File-Name-Separator (default: /) an den Pfad p
 - ***p.filename()*** gibt den Dateiteil des Pfades zurück
 - ***p.stem()*** gibt den Dateinamen ohne Dateierweiterung des Pfades zurück
 - ***p.extension()*** gibt die Dateierweiterung des Pfades zurück



```
#include <filesystem>
```

```
void test(path p) {  
    if (is_directory(p)) {  
        std::cout << p << ":\n";  
        for (const directory_entry& x : directory_iterator{ p }) {  
            const path& f = x; // Greife auf den Pfad des Elements zu  
            if (f.extension() == ".exe")  
                std::cout << f.stem() << " is a Windows executable\n";  
            else {  
                std::string n = f.extension().string();  
                if (n == ".cpp" || n == ".C" || n == ".cxx")  
                    std::cout << f.stem() << " is a C++ source file\n";  
            }  
        }  
    }  
}
```



Dateien und Verzeichnisse

■ Alle File System Operationen besitzen 2 Varianten

- **Default:** Bei einem Fehler wird eine Exception geworfen
- Mit einem extra ***error_code*** Argument *e* wird ein möglicher Fehler in *e* gespeichert
- Error Codes bieten sich an, wenn im normalen Ablauf Fehler häufig erwartet werden

File System Operationen (unvollständig)

p, p1, p2 sind Pfade; *e* ist ein *error_code*; *b* ist ein boolescher Wert; *s* ist ein *std::size_t*

<code>b = exists(p)</code>	Existiert das durch den Pfad referenzierte Objekt?
<code>copy(p1,p2)</code>	Kopiere Dateien und Verzeichnisse von <i>p1</i> zu <i>p2</i>
<code>copy(p1,p2,e)</code>	Kopiere Dateien und Verzeichnisse von <i>p1</i> zu <i>p2</i> ; Speichere Fehler als Fehlercode in <i>e</i>
<code>b = copy_file(p1,p2)</code>	Kopiere den Dateiinhalt von <i>p1</i> zu <i>p2</i>
<code>b = create_directory(p)</code>	Erzeuge das neue Verzeichnis <i>p</i> ; Alle Zwischenverzeichnisse müssen existieren
<code>b = create_directories(p)</code>	Erzeuge das neue Verzeichnis <i>p</i> und alle notwendigen Zwischenverzeichnisse
<code>p = current_path()</code>	<i>p</i> ist das aktuelle Arbeitsverzeichnis
<code>current_path(p)</code>	Ändere das Arbeitsverzeichnis zu <i>p</i>
<code>s = file_size(p)</code>	<i>s</i> ist die Anzahl Bytes in <i>p</i>
<code>b = remove(p)</code>	Entferne <i>p</i> , sofern es eine Datei oder ein leeres Verzeichnis ist

Dateien & Verzeichnisse

Zusammenfassung



- **File Streams** stellen eine Schnittstelle zwischen Dateien auf einem Speichermedium und dem Arbeitsspeicher dar.
 - File Streams müssen mit einer Datei verknüpft sein.
 - Eine verwendete Datei muss auch wieder geschlossen werden, um die Ressource anderen Anwendungen wieder freizugeben.
 - Wenn der File Stream direkt mit einer Datei initialisiert wird, wird die Datei automatisch geschlossen.
- File Stream **Öffnungs-Modi** spezifizieren wie mit der Datei und ihrem Inhalt umgegangen werden soll.
- Die **File System Bibliothek** ermöglicht die Arbeit mit Dateien und Verzeichnissen unabhängig vom darunter liegenden Betriebssystem.

Dateien & Verzeichnisse

Verständnisfragen

PIN: 8008135

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Beim Schließen einer Datei werden die Zeichen des verknüpften Ausgabe-Puffers geflushed. Das heißt, sie werden gelöscht, um den Puffer zu leeren.
- Die aktuelle Lese- oder Schreibposition in einer Datei kann nicht verändert werden.
- Der Typ `std::filesystem::path` dient zur Arbeit mit Dateipfaden.

■ Wozu werden File-Open-Modes genutzt?

- Um eine Datei als Binärdatei zu interpretieren.
- Um beim Öffnen der Datei ans Dateiende zu springen.
- Um sicherzustellen, dass die Datei wieder geschlossen wird.
- Um parallelisiertes Schreiben in eine Datei zu ermöglichen.



Formatierung

Lernziele

- Unterschiedliche Möglichkeiten zur formatierten Ausgabe von Variablen sind bekannt.
 - `std::format` kann als C++ Alternative zu `printf` verwendet werden.
- Reguläre Ausdrücke wurden verstanden und die Syntax ist bekannt.
 - Eigene Reguläre Ausdrücke können zur Verarbeitung von Zeichenketten verwendet werden.



Formatierung

Output Formatierung

- Mit sogenannten Manipulatoren aus den meisten Stream-Bibliotheken (und *<iomanip>*) lassen sich bereits einfache Formatierungen bewerkstelligen.
 - Mit ***std::hex***, ***std::oct*** und ***std::dec*** wird die Zahlenbasis festgelegt, mit alle Ganzzahlwerte bis zum nächsten Manipulator ausgegeben werden sollen.
 - Für Gleitkommazahlen kann die Ausgabe durch ***std::scientific***, ***std::hexfloat***, ***std::fixed*** und ***std::defaultfloat*** manipuliert werden.
 - Bei ***std::fixed*** kann die maximale Anzahl Nachkommastellen über ***std::cout.precision()*** festgelegt werden

```
std::cout << 1234 << '\n'           1234
    << std::hex << 1234 << '\n'      4d2
    << std::oct << 1234 << '\n';      2322
std::cout << 1234 << '\n'           2322
    << std::dec << 1234 << "\n\n";    1234
double d = 123.45678;
std::cout << d << '\n'              123.457
    << std::scientific << d << '\n'  1.234568e+02
    << std::hexfloat << d << '\n'     0x1.edd3be22e5de1p+6
    << std::defaultfloat << d << '\n'  123.457
    << std::fixed << d << "\n\n";     123.456780
std::cout.precision(4);
std::cout << d << '\n'              123.4568
    << 6.54 << '\n'                  6.5400
    << 1 << '\n'                      1
    << 1.0 << "\n\n";                1.0000
std::cout.precision(8);
std::cout << d << '\n'              123.45678000
    << 6.54 << '\n'                  6.54000000
    << 1 << '\n'                      1
    << 1.0 << '\n';                  1.00000000
```

Formatierung

C++20 std::format



```
#include <format>
```

- *std::format* für formatierte Strings mit nachfolgenden Argumenten ähnlich wie *printf()*
 - *std::format* ist typsicher, *printf()* nicht.
 - Der formatierte Teil des Strings wird mit `{` und `}` gekennzeichnet, an dessen Stelle das nächste Argument in den String eingefügt wird.
 - Der leere String `{}` nimmt das nächste Argument aus der Argumentliste und fügt es wie mit `<<` dem String hinzu.
 - Eine Formatierungsanweisung wird mit `:` eingeleitet. Die Basis einer Ganzzahl kann mit **x** (hexadezimal), **o** (oktal), **d** (dezimal) und **b** (binär) festgelegt werden.

```
auto val1 = "Welt";  
auto val2 = 2024;  
std::cout << std::format("Servus {}\n", val1);  
std::cout << std::format("Servus {}\n", val2);  
  
std::cout << std::format("{} {:x} {:o} {:d} {:b}\n",  
    1234, 1234, 1234, 1234, 1234);
```

```
Servus Welt  
Servus 2024  
1234 4d2 2322 1234 10011010010
```

Formatierung

C++20 std::format



```
#include <format>
```

- *std::format* für formatierte Strings mit nachfolgenden Argumenten
 - Eine Formatierungsanweisung wird mit **:** eingeleitet. Die Basis einer Ganzzahl kann mit **x** (hexadezimal), **o** (oktal), **d** (dezimal) und **b** (binär) festgelegt werden.
 - Gleitkommazahlen können wie bei Output Streams mit den Bezeichnern **e** (scientific), **a** (hexfloat), **f** (fixed), **g** (default) ausgegeben werden. Ein Punkt leitet die Präzisionsangabe ein.
 - Mit einer Nummer vor dem Doppelpunkt kann explizit angegeben werden, welches Argument aus der Liste verwendet werden soll.

```
std::cout << std::format("{3:} {1:x} {2:o} {0:b}\n", 000, 111, 222, 333);  
std::cout << std::format("{0:}; {0:e}; {0:a}; {0:f}; {0:g}\n", 123.45678);  
std::cout << std::format("Precision(8): {:.8} {:.8}\n", 123.45678, 6.54f);  
std::cout << std::format("Precision(4): {:.4} {:.4}\n", 123.45678, 6.54f);  
std::cout << std::format("Precision(4): {:.4e}; {:.4e}; {:.4e};\n", 123.4567, 6.54f, 0.000512);
```

```
333 6f 336 0  
123.45678; 1.234568e+02; 1.edd3be22e5de1p+6; 123.456780; 123.457  
Precision(8): 123.45678 6.54  
Precision(4): 123.5 6.54  
Precision(4): 1.2346e+02; 6.5400e+00; 5.1200e-04;
```


Formatierung

C++20 std::format



```
#include <format>
```

- *std::format* für formatierte Strings mit nachfolgenden Argumenten
 - Zusammen mit der Anzahl Zeichen, die ein Argument einnehmen soll, kann die Ausrichtung des Arguments festgelegt werden: < für linksbündig, ^ für zentriert und > für rechtsbündig
 - Das Zeichen zwischen dem Doppelpunkt und der Ausrichtung repräsentiert das Füllzeichen
 - Die Funktion **std::vformat()** nimmt eine Variable als Formatangabe für mehr Flexibilität und mögliche Laufzeitfehler

```
std::cout << std::format("{:-^10}\n{:~10}\n\n", " Servus ", " Welt ");  
char fill = ' ';  
unsigned length = 8;  
std::string numberFormat = "{:s + fill + ">" + std::to_string(length) + "b}";  
std::cout << std::vformat(numberFormat + '\n' + numberFormat + '\n' + numberFormat + "\n\n"  
                        , std::make_format_args(10,216,58));  
  
fill = '0';  
length = 9;  
numberFormat = "{:s + fill + ">" + std::to_string(length) + "b}";  
std::cout << std::vformat(numberFormat + '\n' + numberFormat + '\n' + numberFormat  
                        , std::make_format_args(10, 216, 58));
```

```
- Servus -  
-- Welt --  
  
      1010  
11011000  
      111010  
  
000001010  
011011000  
000111010
```

Reguläre Ausdrücke

Sonderzeichen



```
#include <regex>
```

- Gängiges Werkzeug zur Verarbeitung von Text
 - Beschreiben Patterns in Texten (z.B. Aufbau von IBAN-Nummern, Datumsangaben)
 - Geeignet, um solche Patterns in Texten zu finden
- Wird in C++ durch den `<regex>` Header (Regular Expressions) unterstützt
 - Unterstützt verschiedene Notationsstile
 - Default Stil ist die Variante des ECMA-Standards, der auch für JavaScript verwendet wird
 - Syntax von regulären Ausdrücken basiert auf Sonderzeichen:

Reguläre Ausdrücke Sonderzeichen					
.	Ein beliebiges Zeichen („Wildcard“)	\	Leitet Sonderzeichen ein	*	Beliebig häufig (Suffix)
[...]	Character-Klasse		Alternative / Oder	+	Mindestens einmal (Suffix)
{...}	Anzahl der Vorkommen	^	Zeilenanfang; Negierung	?	Maximal einmal (Suffix)
(...)	Gruppierung	\$	Zeilenende		

Reguläre Ausdrücke

Sonderzeichen



■ Beispiel: $^A^*B+C? \$$

- Beliebig viele As am Anfang, gefolgt von mindestens einem B und maximal einem C am Zeilenende
- Übereinstimmungen:
 - AAAAAAAAAAAAAABBBBBBBBBBBBBBBBC
 - BC
 - B
- Keine Übereinstimmung:
 - AAAAAAA
 - AAAABC
 - AABBC

■ Beispiel: $A\{3\}B\{2,4\}C\{0,\}$

- Genau 3 As, 2 bis 4 Bs und beliebig viele Cs
- Übereinstimmungen:
 - AAABBC ; AAABBB
- Keine Übereinstimmung:
 - AABBC ; AAABC ; AAABBBBBBBCC

Reguläre Ausdrücke Sonderzeichen

.	Ein beliebiges Zeichen („Wildcard“)
[...]	Character-Klasse
{...}	Anzahl der Vorkommen
(...)	Gruppierung / Subpatterns
\	Leitet Sonderzeichen ein
	Alternative / Oder
^	Zeilenanfang; Negierung
\$	Zeilenende
*	Beliebig häufig (Suffix)
+	Mindestens einmal (Suffix)
?	Maximal einmal (Suffix)

Reguläre Ausdrücke

Zeichenklassen



- Zeichenklassen umfassen bestimmte Arten von Zeichen

Zeichenklassen			
Klasse	Bedeutung	Verwendung	Alternative
alnum	Ein alphanumerisches Zeichen	<code>[:alnum:]</code>	
alpha	Ein Buchstabe	<code>[:alpha:]</code>	
blank	Leerzeichen (kein Zeilenumbruch)	<code>[:blank:]</code>	
s space	Leerzeichen	<code>[:space:]</code>	<code>\s</code> Oder Verneinung: <code>\S</code>
cntrl	Kontrollzeichen	<code>[:cntrl:]</code>	
d digit	Dezimalzeichen	<code>[:digit:]</code>	<code>\d</code> Oder Verneinung: <code>\D</code>
graph	Graphisches Zeichen	<code>[:graph:]</code>	
lower	Kleinbuchstabe	<code>[:lower:]</code>	<code>\l</code> Ist kein Standard!
upper	Großbuchstabe	<code>[:upper:]</code>	<code>\u</code> Ist kein Standard!
print	Druckbare Zeichen (z.B. kein Tab)	<code>[:print:]</code>	
punct	Satzzeichen	<code>[:punct:]</code>	
w	Wortzeichen (alnum und <code>_</code>)	<code>[_:alnum]</code>	<code>\w</code> Oder Verneinung: <code>\W</code>
xdigit	Hexadezimalziffer	<code>[:xdigit:]</code>	

Reguläre Ausdrücke

Beispiele



Beispiele			
Regulärer Ausdruck	Bedeutung	Beispiele	Gegenbeispiele
Ax+	A gefolgt von mindestens einem x	„Ax“ „Axxx“	„“ „AxAx“ „A“
(Ax)+	Mindestens einmal Ax	„Ax“ „AxAx“	„“ „Axxx“ „A“
\d-?\d	Zwei Ziffern, optional durch - getrennt	„1-2“ „12“	„1--2“ „-2“
\w{2}-\d{4,5}	Zwei Zeichen, gefolgt von 4-5 Ziffern	„Ab-1234“ „_X-54321“ „22-4321“	„5%-1234“
(bs BS)	bs komplett klein oder groß	„bs“ „BS“	„bS“ „Bs“
b(s B)S	Zwischen b und S kommt s oder B	„bsS“ „bBS“	„bS“ „bsBS“
[aeiou]	Ein deutscher Vokal	„a“ „e“ „i“ „o“ „u“	„x“ „y“
[^aeiou]	Kein deutscher Vokal	„x“ „y“ „k“	„e“
[a^eiou]	Ein deutscher Vokal oder ^	„a“ „e“ „^“	„*“ „x“
<(.*?)>(.*?)</\1>	XML Befehl. \1 referenziert die erste Gruppe	„<Name>Bernd</Name>“	„<n>Hi</y>“
DEd{2}(\s?\d{4}){4}\s?d{2}	Deutsche IBAN (2 Buchstaben, 2 Ziffern, 4*4 Ziffern, 2 Ziffern)	„DE11 9900 0000 8888 7777 00“ „DE11990000008888777700“	„DE 1100 00“

Reguläre Ausdrücke

Anwendung



```
#include <regex>
```

- Funktionen der STL zur Verwendung von regulären Ausdrücken:
 - `regex_match()`: Überprüft, ob ein String mit dem regulären Ausdruck übereinstimmt
 - `regex_search()`: Sucht nach einem Substring, der mit dem RegEx übereinstimmt
 - `regex_replace()`: Tauscht den Substrings aus, der mit dem RegEx übereinstimmt
 - `regex_iterator`: Iteriert über alle Substrings, die mit dem RegEx übereinstimmen
 - `regex_token_iterator`: Iteriert über alle Substrings, die nicht mit dem RegEx übereinstimmen
- Für die Definition von RegEx empfiehlt es sich, „**Raw Strings**“ zu verwenden
 - Starten mit `R"("` und enden mit `)"`
 - Erlauben, Backslash und Anführungszeichen ohne extra Backslash zu verwenden

```
std::regex reg{ R"((\w{2})-(\d{4,5}))" };  
std::string s{ "54321X-4235Ab21" };  
std::smatch fund; // Datentyp für übereinstimmende Strings  
if (std::regex_search(s, fund, reg))  
    std::cout << fund[0] << ": subReg1: " << fund[1] << " subReg2: " << fund[2];
```

std::smatch :

[0] enthält übereinstimmenden String

[>0] enthält Substrings der möglichen Sub-RegEx

Ausgabe:

```
1X-4235: subReg1: 1X subReg2: 4235
```

Reguläre Ausdrücke

RegEx Iteratoren



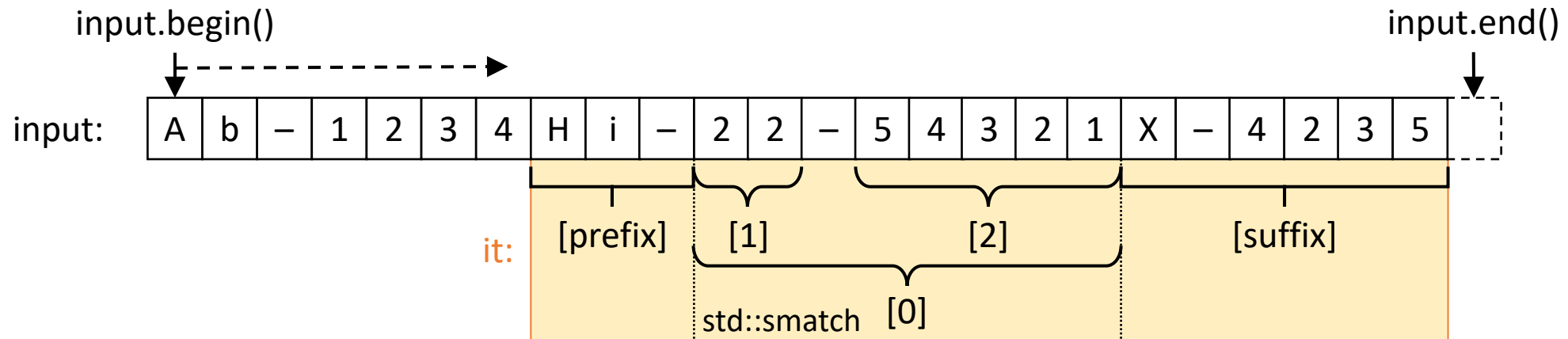
```
#include <regex>
```

- Um nicht nur den ersten Substring zu finden, der mit einem regulären Ausdruck übereinstimmt, bieten sich Iteratoren an.
 - RegEx Iteratoren zeigen auf ein `std::smatch` Objekt und enthalten zusätzliche Infos
 - Durch Inkrementieren des RegEx Iterators zeigt dieser auf das nächste Objekt
 - Wenn der RegEx Iterator leer ist, existieren keine weiteren übereinstimmenden Substrings

```
std::regex reg{ R"((\w{2})-(\d{4,5}))" };  
std::string input{ "Ab-1234Hi-22-54321X-4235" };  
for (std::sregex_iterator it(input.begin(), input.end(), reg); it != std::sregex_iterator{}; ++it)  
    std::cout << (*it)[0] << '\n';
```

Ausgabe:

```
Ab-1234  
22-54321
```



Formatierung

Zusammenfassung



- Für die Ausgabe können mithilfe von **Stream Manipulatoren** einfache Formatierungen getätigt werden.
- Mithilfe von ***std::format*** können Inhalte ähnlich wie mit *printf* für die Ausgabe unterschiedlich formatiert werden.
 - *std::format* ist typsicher, *printf* ist nicht typsicher
- **Reguläre Ausdrücke** werden verwendet, um bestimmte Regelmäßigkeiten / Muster in Texten zu finden.

Formatierung

Verständnisfragen

PIN: 8008135

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Mit "*fixed*" und "*scientific*" kann man das Format von Gleitkommazahlen steuern.
- Mit "*regex_search*" lässt sich feststellen, ob ein Teil eines Strings einem Muster entspricht.

■ Welche Angaben erfüllen den regulären Ausdruck `\w+\s\d{4}`?

- Apfel 2023
- Kaktus+1234
- Orange2022
- 4321 1234
- 2023 Apfel
- Erdb33re 7777
- Haus 12
- Banane 3*21



- Die STL I/O-Streams sind typsicher, typsensitiv und erweiterbar
 - C-Style I/O ist **nicht** typsicher
- Streams können nicht kopiert werden, es benötigt Move-Operationen
- Bedenke immer fehlerhafte Eingaben beim Lesen
- Überschreibe die Eingabe- (>>) und Ausgabeoperatoren (<<) für deine eigenen Klassen mit informativen Repräsentationen in Text
- >> überspringt standardmäßig Leerzeichen am Anfang
- Nutze den fail-Status, um behebbare I/O-Fehler abzuhandeln
- Nutze Manipulatoren oder *std::format()* für Formatierungen
- Das Standard Format kann auch Zeitstempel und Daten ausgeben
- Überprüfe vor der Verwendung, ob eine Datei mit einem File Stream verknüpft ist
- Nutze Reguläre Ausdrücke bedacht und für konventionelle Probleme
- Nutze *regex_iterator* zur Suche nach Patterns bei Streams